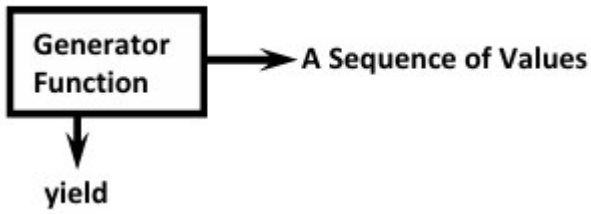
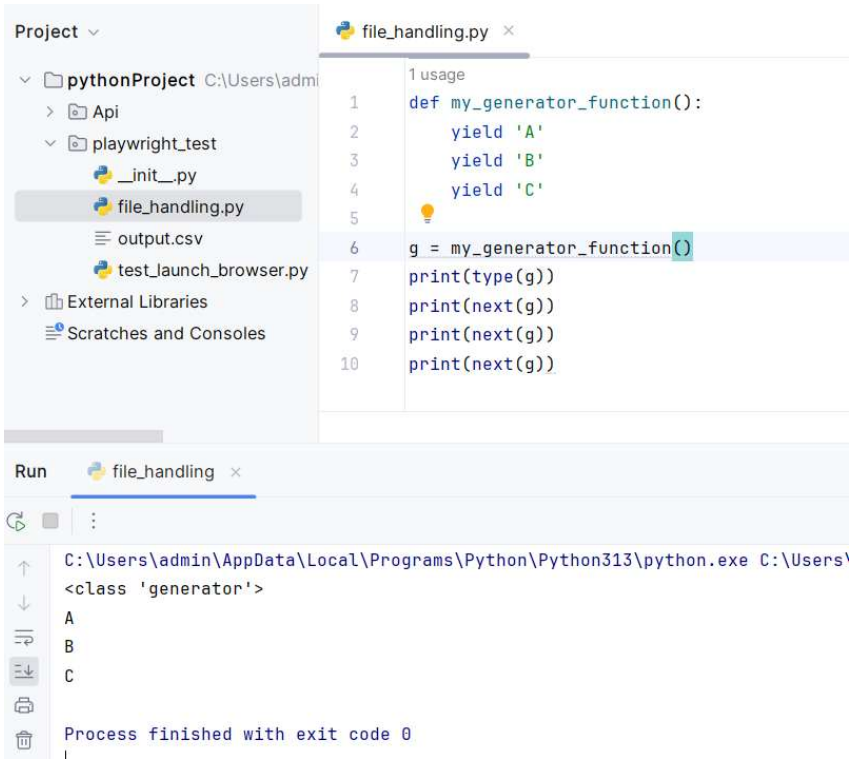
Generator

Generator is a function which is responsible to generate a sequence of values.

We can write generator functions just like ordinary functions, but it uses yield keyword to return values.



A **Generator** is a special type of function in Python that allows you to return a sequence of values **one at a time**, instead of all at once. It uses the `yield` keyword instead of `return`.



In Python, the next() function is used to manually retrieve the **next item** from an iterator (like a generator, list iterator, or file object).

Compare a normal function (list) vs a generator.

Normal Function

```
def get_squares(n):
    result = []
    for i in range(n):
        result.append(i * i)
    return result # Returns the WHOLE list at once
```

Generator Function

```
def get_squares_gen(n):
    for i in range(n):
        yield i * i # Yields one square at a time

# Using it in a loop
for num in get_squares_gen(5):
    print(num)
```

Using Comprehension [List & Tuple]

```
# List comprehension (Square brackets []) - Creates the whole list in memory
my_list = [x*x for x in range(1000000)]

# Generator expression (Parentheses ()) - Creates a generator object
my_gen = (x*x for x in range(1000000))

print(next(my_gen)) # 0
print(next(my_gen)) # 1
```

Summary Table

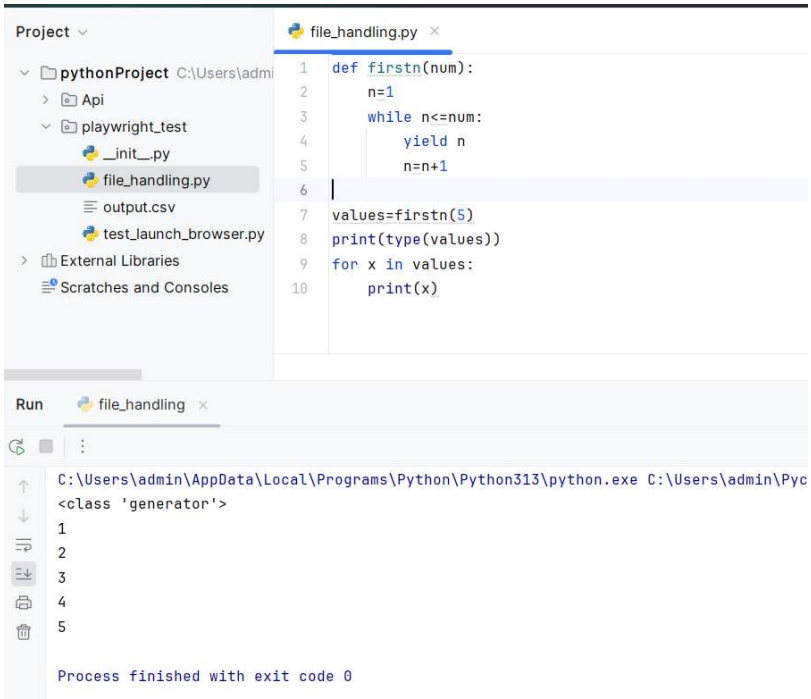
Feature	List / Function	Generator
Keyword	`return`	`yield`
Memory	High (stores all items)	Low (stores one item)
Performance	Slower for large data	Faster for large data
Access	Can index (`list[0]`)	Only sequential (`next()`)

Advantages of Generator Functions:

1) When compared with Class Level Iterators, Generators are very easy to use.
2) Improves Memory Utilization and Performance.
3) Generators are best suitable for reading Data from Large Number of Large Files.
4) Generators work great for web scraping and crawling.

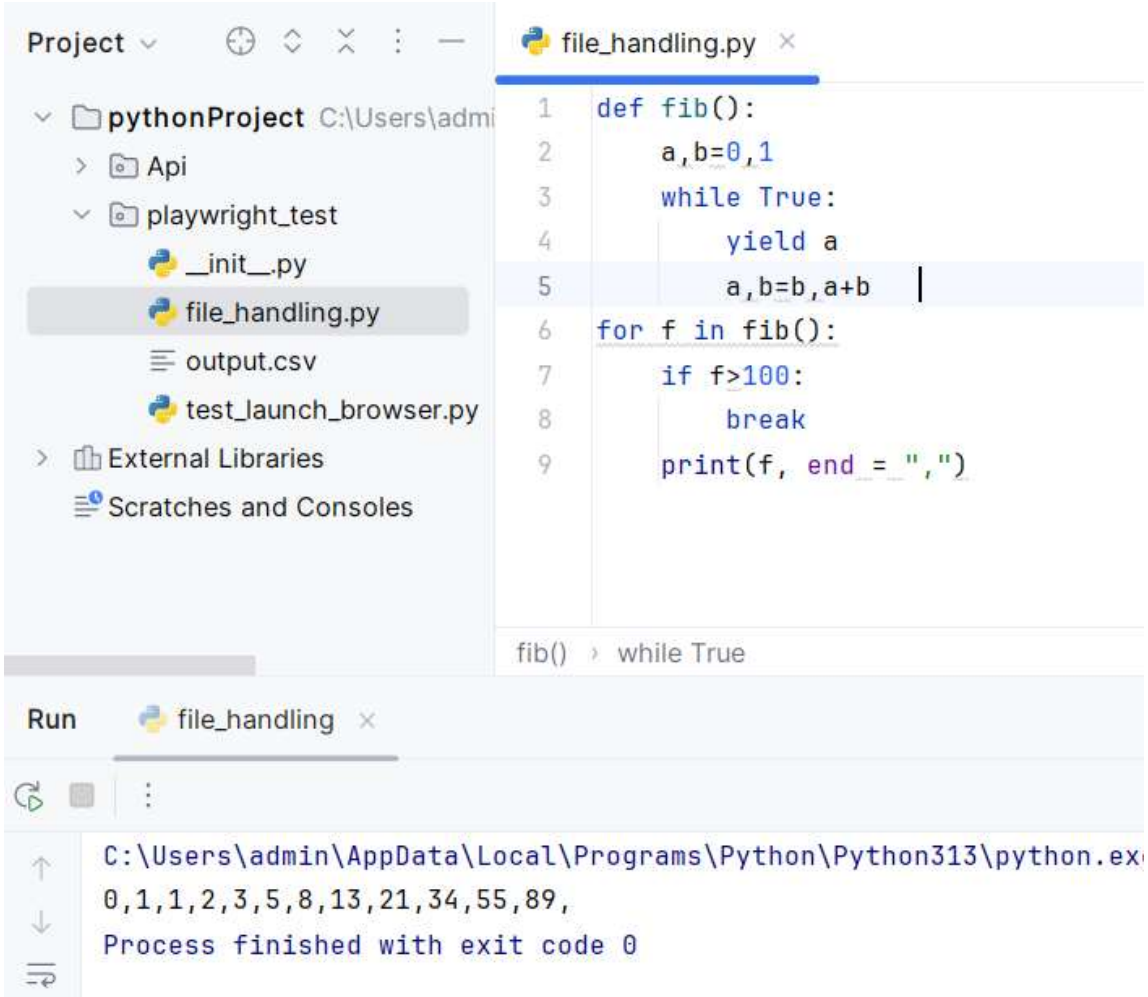
Ques

To generate first n numbers?



Ques

To generate Fibonacci Numbers?



Ques

Difference between generator and iterator?

Feature	Iterator	Generator
Creation	Created using `iter()` or by defining a **Class** with `__next__`.	Created using a **Function** with the `yield` keyword.
Complexity	Requires more code (writing a whole class).	Very simple (just a function).
State	You have to manually track the state (variables) in the class.	Python **automatically** saves the state (local variables) for you.

- **Iterator:** A general concept (protocol). Any object that has the `__next__()` and `__iter__()` methods is an iterator. You usually create them by calling `iter()` on a list or string.
- **Generator:** A specific, easy way to *create* an iterator using a **function** and the **`yield`** keyword.

DECORATORS

Decorator is a function which can take a function as argument and extend its functionality and returns modified function with extended functionality.

Ex 1

```
def my_decorator(func):
    def wrapper():
        print("Something is happening before the function is called.")
        func()
        print("Something is happening after the function is called.")
    return wrapper

@my_decorator
def say_hello():
    print("Hello!")

say_hello()
```

Ex 2

```
def wish(name):
    print("Hello", name, "Good Morning")
```

```
wish("Ravi")
```

Hello Ravi Good Morning

```python
def decor(func):
    def inner(name):
        if name == "Prateek":
            print("Hello Prateek Bad Morning")
        else:
            func(name)
    return inner


@decor
def wish(name):
    print("Hello", name, "Good Morning")


wish("Prateek") → Hello Prateek Bad Morning
wish("Ravi")   →   Hello Ravi Good Morning
wish("Sunny") →   Hello Sunny Good Morning
```

How to call Same Function with Decorator and without Decorator:

decorfunction=decor(wish)

wish("Prateek") #decorator wont be executed

wish("Sunny") #decorator wont be executed

decorfunction("Prateek") #decorator will be executed

decorfunction("Sunny") #decorator will be executed

Ques

Division function using decorator

```
def division(a,b):
    return a/b
```

division(10/2) → 5.0

division(10/0) → Exception (zero division error)

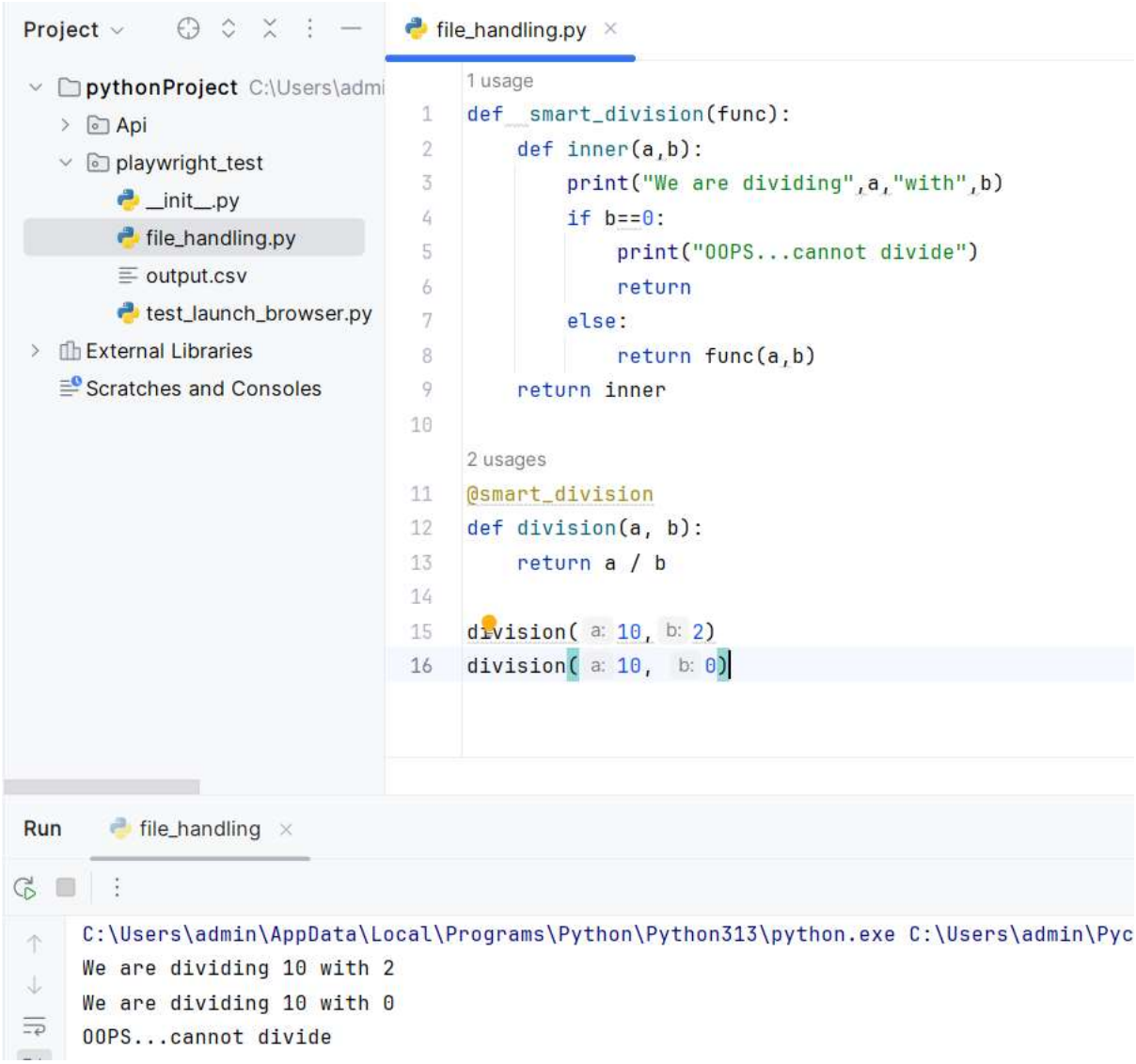
Solving 10/0 exception using decorator

```
def  smart_division(func):
    def inner(a,b):
        print("We are dividing",a,"with",b)
        if b==0:
            print("OOPS...cannot divide")
            return
        else:
            return func(a,b)
    return inner



@smart_division
def division(a, b):
    return a / b

division(10,2)
division(10, 0)
```



In Object-Oriented Programming (OOP), decorators are used to define the **behavior of methods** within a class. The most common built-in decorators are `@classmethod`, `@staticmethod`, `@property` and `@abstractmethod`

Ques:

```
def calculator(a,b:
        Return a+b
```

Above function is only doing SUM. Write a decorator function to extend the functionality as sum, difference, multiply, division.

```
def extended_calc(func):
    def wrapper(a, b):
        # Perform extra operations
        print(f"Difference: {a - b}")
        print(f"Product:    {a * b}")
        try:
            print(f"Division:   {a / b}")
        except ZeroDivisionError:
            print("Division:   Cannot divide by zero")

        # Call the original function (which handles Sum)
        return func(a, b)
    return wrapper

@extended_calc
def calculator(a, b):
    return a + b

# --- Testing the code ---
result = calculator(10, 5)
print(f"Sum:         {result}")
```